How to optimize low-power modes on STM32N6 MCUs

Introduction

This application note deals with the use of low-power modes on the STM32N6 devices. It is applicable to all the devices of the STM32N6 series.

The STM32N6 devices are based on an Arm® Cortex®-M55 CPU using 16 nm FinFET technology. It can run up to 800 MHz (overdrive mode).

The STM32N6 devices embed the Neural-ART accelerator, an NPU with its subsystems, which is designed to accelerate deep learning inference for AI applications. The NPU can run up to 1 GHz.

They have an internal SMPS. With their built-in internal voltage regulator and voltage scaling, the consumption in active modes is kept at a minimum. The multivoltage domains of the device permit the core to be energized at a low voltage of 0.8 V, while the I/Os and analog components can function at a higher supply voltage of 1.8 V or 3.3 V.

They support a backup domain, which can be supplied by a battery to keep the RTC running. The backup domain encompasses a collection of 32 registers, each of which is 32 bits wide, as well as 8 Kbytes of backup SRAM that can be accessed in 32-, 16-, or 8-bit data mode. This SRAM can be retained even in the event that the MCU experiences a loss of the primary power supply. The backup battery can be charged when the main supply is plugged in.

The STM32N6 devices have low-power capabilities to reduce the power consumption. They support different low-power modes that can achieve the best compromise between low-power consumption, shorter startup/wake-up time, and the number of active peripherals.

For further information on STM32N6, refer to Related documents and tools.

1 General information

The Arm® Cortex®-M55 is based on the Arm®v8.1-M architecture. This architecture includes Arm® TrustZone®, FPU, DSP, and Arm® Helium™ technology. Arm® Helium™ technology is an M-profile vector extension (MVE).

Note: Arm and TrustZone are registered trademarks of Arm Limited (or its subsidiaries) in the US and/or elsewhere.

arm

Related documents and tools

[1] STM32N6 reference manual (RM0486)

[2] Getting started with STM32N6 MCU hardware development (AN5967)

[3] STM32N647xx/STM32N657xx datasheet (DS14791)

2 Glossary

Table 1. **Glossary**

Term	Meaning
ADC	Analog-to-digital converter
AHB	Advanced high-performance bus
BKPSRAM	Backup SRAM
CIC	Cascaded integrator-comb filter
CPU	Central processing unit
CSI	Camera serial interface
DMA	Direct memory access
DPS	Dynamic power switching
D-TCM	Data tightly coupled memory
DSP	Digital signal processing
DVFS	Dynamic voltage and frequency scaling
EXTI	External interrupt/event
FPU	Floating-point unit
GPIO	General-purpose input/output
HAL	Hardware abstraction layer
HDP	Hardware debug port
HPF	High-pass filter
HSI	High-speed internal (oscillator)
HSE	High-speed external (oscillator)
HSIS	High-speed internal secure (oscillator)
I-TCM	Instruction tightly coupled memory
IRQ	Interrupt request
IWDG	Independent watchdog
LSE	Low-speed external clock (oscillator)
MCU	Microcontroller unit
MDF	Multi-function digital filter
MDPS	Memory power switching
MSI	Multi-speed internal clock
MUX	Multiplexer
MVE	M-profile vector extension
NPU	Neural processing unit/ST Neural-ART accelerator
NVIC	Nested vector interrupt controller
OTP	One-time-programmable (bytes)
PDR	Power down reset
PLL	Phase locked loop
PWR	Power control block
RCC	Reset and clock control
RSFLT	Reshape filter

Term	Meaning
RTC	Real-time clock
SDMMC	Secure digital and multimedia card interface that supports SD, MMC, eMMC, and SDIO protocols.
SMPS	Switched-mode power supply
SRAM	Static random-access memory
TEMP	Temperature sensor
UCPD	USB Type-C®/ power delivery
USART	Universal synchronous/asynchronous receiver transmitter
USB	Universal serial bus
UART	Universal asynchronous receiver/transmitter
VCO	Voltage-controlled oscillator
Volt. Mon.	Voltage monitor
WFE	Wait for event
WFI	Wait for interrupt
XSPI	Extended serial peripheral interface
XSPIM_P1	XSPI manager Port 1
XSPIM_P2	XSPI manager Port 2

3 STM32N6 power supply

This section describes the STM32N6 power supply.

3.1 System architecture

The STM32N6 devices have a complex system architecture. This architecture is different from the other STM32 MCU architectures that just need V_{DD} to start up. Multiple voltage supplies must be applied with a certain sequence to start the devices. For further information, consult the document [2].

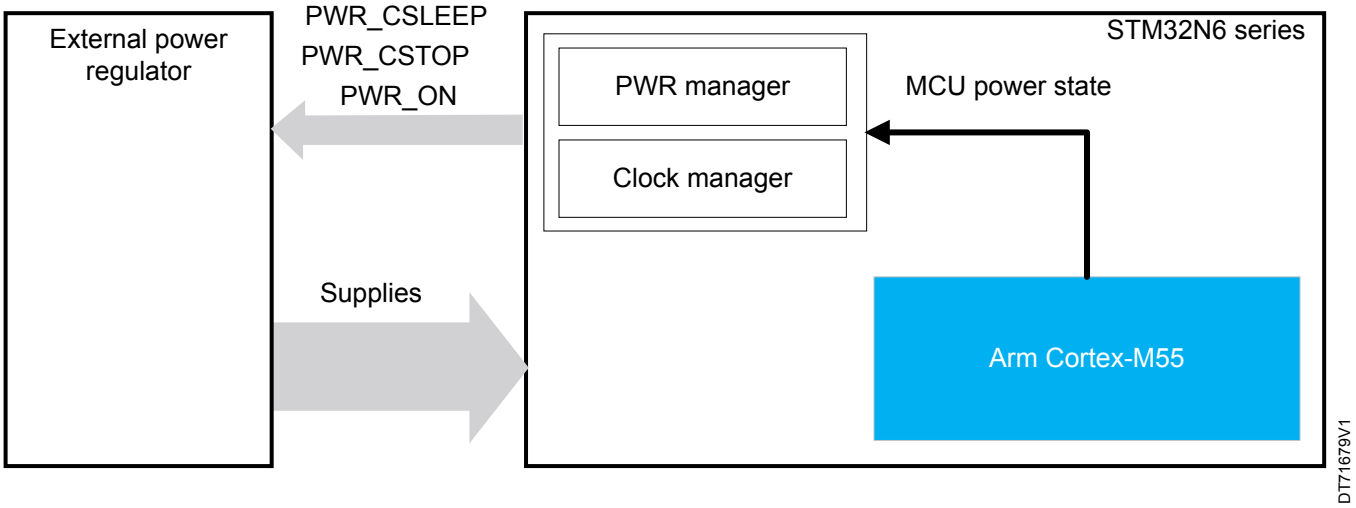
The power management features are spread between the RCC (reset and clock control) and PWR (power) blocks.

The RCC block ensures the clock tree handling (PLLs, multiplexers, dividers, clock gating) and the resets (local resets of the peripherals, Cortex®-M55 reset, resets from PWR).

The PWR block is responsible for a low-power entry/exit. The request to enter the low-power mode comes from the CPU and is sent to the RCC. The RCC block cuts off the clock or gates the clock according to the mode. Then, the RCC asks the PWR to enter the requested low-power mode. For the wake-up, a request comes from EXTI or some peripherals is sent to the PWR. The PWR restores the supply and informs the RCC that can consequently turn on the clock. In an internal SMPS configuration, the mechanism that restores power supply is done internally. On the other hand, in an external SMPS configuration, the PWR_ON signal is used to inform the external power regulator to restore the supply.

Figure 1 shows the STM32N6 high-level system architecture.

Figure 1. High-level system architecture



3.2 System supply

The system requires an external supply on the VDD, VDDA18AON, and VDDCORE pins to start and allows independent supplies for the following components:

- PLLs, temperature sensor, voltage monitoring (VDDA18PLL pin)
- USB, UCPD (VDD33USB pin, VDDA18USB pin)
- ADC (VDDA18ADC pin, VREF+ pin, VREF-pin)
- CSI (VDDA18CS pin, VDDCSI pin)
- XSPIM1 -16-bit interface external memory (VDDIO2 pin)
- XSPIM2 - 8-bit interface external memory (VDDIO3 pin)
- SDMMC1 (VDDIO4 pin)
- SDMMC2 (VDDIO5 pin)
- Backup domain in Standby mode (VBAT pin)

Refer to the document [3] for every voltage value of the power pin.

The V_{DDCORE} supplies all the digital circuitries except for the backup domain and the retention domain in Standby mode. It can be directly supplied by an external power source or using the internal SMPS step-down converter.

The $V_{DDA18AON}$ supplies the system reset, the power management, the oscillators (HSE, HSI, HSIS), and the OTP memory.

The V_{DD} supplies the I/Os. The $V_{DDA18AON}$ and V_{DD} must be supplied before V_{DDCORE} following the sequence described in section 2.6 *System startup* in the document [2].

The internal MCU system is supplied through different power domains: core domain, retention domain, and backup domain.

Core domain

The Core domain includes CPU, NPU, interconnection, peripherals, RAM, and is supplied by V_{DDCORE}.

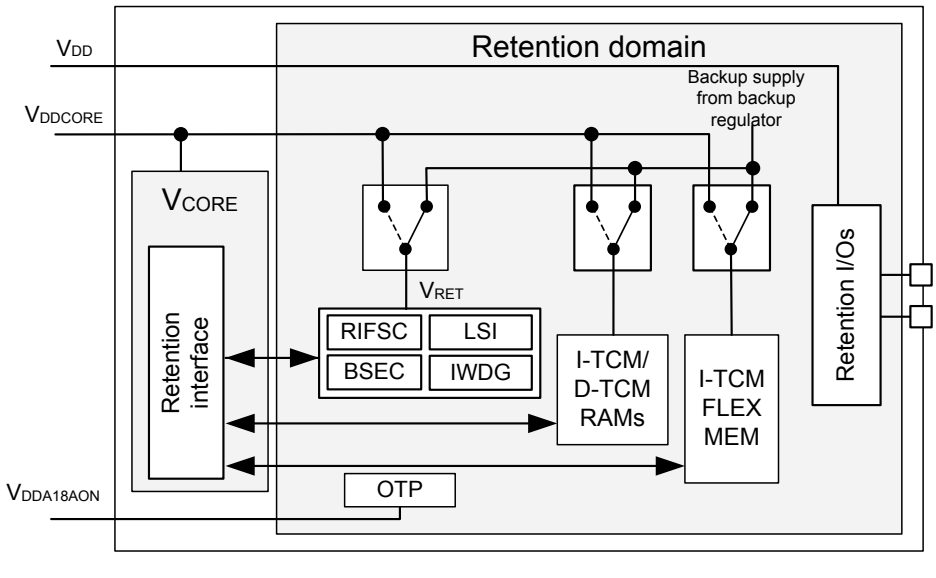
Refer to the section *Core domain* in the document [1] for more information.

Retention domain

The retention domain (Figure 2) includes boot, OTP controller (BSEC), independent watchdog (IWDG), LSI, resource isolation framework controller (RIFSC), I-TCM, D-TCM, and I-TCM FLEXRAM.

The content of this domain is retained in Standby mode and is not powered in VBAT mode. Refer to the section *Retention domain* in the document [1] for more information.

Figure 2. Retention domain

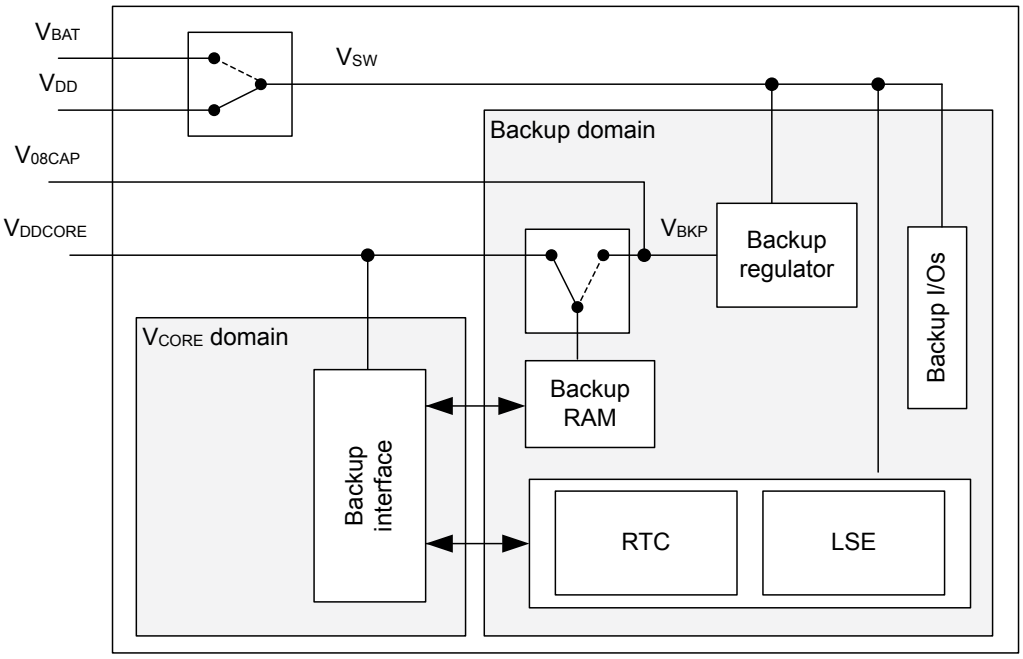


Backup domain

The backup domain includes 8 Kbytes of BKPSRAM, backup registers, and RTC. The backup SRAM content is retained even in Standby and/or VBAT mode (see Figure 3).

For more information, refer to the section *Backup domain* in the document [1].

Figure 3. **Backup domain**



DT70452V1

Analog domain

The analog domain includes an ADC and an internal voltage reference V_{REF}. Note that the analog domain has an isolated supply ground V_{SSA}.

Refer to the section *Analog supply* in the document [1] for more information.

3.3 Power management

In digital circuits, the dynamic power consumption is proportional to the frequency and the voltage squared. A tradeoff must be made since both need to be increased when a performance is needed, and decreased when the target is to save power. Hence, any user running applications can adjust the frequency of the system clock whenever it is necessary and at any time according to its use case. In addition to that flexibility, the STM32N6 devices introduce two features to manage power consumption (dynamic or static):

Voltage scaling is a power management technique. It is a technique, which consists of increasing/reducing the voltage to support a high/low frequency in active mode. Additionally, it allows the operating voltage to be reduced when the MCU is not active.

- The voltage scaling can operate in:
  - Run and Sleep modes (VOS HIGH/LOW)
  - Stop mode (SVOS HIGH/LOW)
- Operating modes:
  - Run (power on, clock on)
  - Sleep (power on, core clock stopped, peripherals kept running)
  - Stop (power on, clock off)
  - Standby (power off, clock off)

The number of MCU features used may impact the power consumption. Indeed, the increased use of MCU peripherals leads to a greater power consumption. Hence, depending on the application, each peripheral and internal memory can be gate clocked in the RCC registers (PERxEN/MEMEN) independently when it is not used.

Some peripherals or/and power domain having independent supply (see Section 3.2: System supply) can be powered down when they are not used. Internal memories have a power switch that users can turn off when not in use (see register *RAMCFG_AXISRAMxCR* in the document [1]).

4 STM32N6 low-power features

This section describes the STM32N6 various low-power modes and the different ways of entering or exiting them.

4.1 Entering/exiting low-power modes

The operating modes of the system enable the control of the clock distribution to the various parts of the system. This ensures that each component receives the appropriate clock signal. Additionally, the operating modes allow for the control of the power supplies to the system, ensuring that each component receives the appropriate voltage.

The power management controls the SMPS step-down converter to supply the V_{DDCORE} in accordance with the system operating modes.

The choice of a low-power mode depends on the power-saving target. It must be chosen according to the wake-up interrupts needed or available in the chosen low-power mode and the expected wake-up time duration.

In general, the deeper the low power-saving mode is, the greater the power consumption reduction is. The Standby mode consumes less than the Stop mode, and the latter consumes less than the Sleep mode.

However, a deeper low power-saving mode leads to a longer wake-up time. Waking up from the Sleep mode is faster than waking up from the Stop mode. It goes without saying that waking up from the Stop mode is then faster than waking up from the Standby mode.

In addition, the user must always make sure that the chosen low-power mode is consistent with the available wake-up sources. For instance, the Standby mode has limited wake-up source capabilities.

Refer to the table *Functionalities depending on system operating mode* in the document [1] for wake-up source capabilities.

4.1.1 Sleep mode

In Sleep mode, the CPU clock is stopped. The CPU subsystem peripheral clocks operate according to the values of PERxLPEN bits in RCC_xxxxLPENR. The MCU can be either on VOS HIGH or VOS LOW by setting or clearing the bit VOS of the PWR_VOSCR register.

Note: All examples provided in this section must be executed in a Secure mode.

For further information, see the section about the Sleep mode in the document [1].

To enter or exit the Sleep mode, refer to the Table 2.

Table 2. Sleep mode

Sleep mode	Description
Mode entry	WFI or WFE while: • SLEEPDEEP = 0(1) • CPU NVIC interrupts and events are cleared.
	On return from ISR while: • SLEEPDEEP = 0 • SLEEPONEXIT = 1(1) • CPU NVIC interrupts and events are cleared.
Mode exit	If WFI or return from ISR was used for entry: • any interrupts enabled in NVIC(2) If WFE was used for entry and SEVONPEND = 0(1): • any event If WFE was used for entry and SEVONPEND = 1: • any interrupt even when disabled in NVIC or any event
Wake-up latency	None

1. *Refer to the Cortex®-M55 system control register(SCR).*
2. *See the table "STM32N6xx vector table" in the document [1] for the list of NVIC connected peripherals*

HAL version

```
int main(void)
{
    SystemInit(); //Sometimes already called in the Startup file
    HAL_Init();
    SystemClock_Config();

    /* User code begin*/
        //etc...
    /* User code end*/

    /*Voltage scaling to VOS HIGH*/
    HAL_PWREx_ControlVoltageScaling(PWR_REGULATOR_VOLTAGE_SCALE0);

    /*Clear interrupts and events to avoid interrupt generation and MCU wake-up
     in Sleep mode.*/

    HAL_SuspendTick(); // Suspend Systick

    /*for example deactivating USART1 Rx/Tx interrupt or clearing any pending
     interrupt*/
        //etc...

    /*Entering sleep mode */
    HAL_PWR_EnterSLEEPMode(0,PWR_SLEEPENTRY_WFI);
     return (0);
}
```

4.1.2 Stop mode

In Stop mode, the CPU clock is stopped. All CPU subsystem peripheral clocks are stopped too, and only the CPU subsystem peripherals having a PERxLPEN bit operate accordingly. The MCU supports SVOS HIGH and SVOS LOW in Stop mode.

The HSI and the MSI can remain enabled in Stop mode (only SVOS HIGH). It is done by setting the bits HSISTOPEN and MSISTOPEN of the RCC_STOPCR register. Therefore, the clocks are available immediately when the system exits the Stop mode, or when a peripheral requests the kernel clock. It goes without saying that the power consumption is impacted accordingly.

Depending on the bit STOPWUCK of the register RCC_CFGR1, the MCU can exit the Stop mode with the selected clock (HSI or MSI).

Warning: *The BSEC CLOCK must be enabled in RCC_APB4HENR register to be able to enter Stop mode.*

Note: All examples provided in this section must be executed in a Secure mode.

For more information, see the Stop mode section in the document [1], chapter *Clock generation in Stop and Standby modes*.

To enter/exit the Stop mode, refer to the Table 3 and Table 4.

Table 3. Stop mode (SVOS high)

Stop mode SVOS high	Description
Mode entry	WFI or WFE while: • SLEEPDEEP = 1(1) • CPU NVIC interrupts and events cleared • All CPU EXTI wake-up sources cleared
	On return from ISR while: • SLEEPDEEP = 1 • SLEEPONEXIT = 1(1) • CPU NVIC interrupts and events cleared

Stop mode SVOS high	Description
Mode entry	• All CPU EXTI wake-up sources cleared
Mode exit	If WFI or return from ISR was used for entry: • EXTI interrupt enabled in NVIC(2) If WFE was used for entry and SEVONPEND = 0: • EXTI event (3) If WFE was used for entry and SEVONPEND = 1: • EXTI interrupt even when disabled in an NVIC(2) or EXTI event
Wake-up latency	SMPS wake-up time from low-power mode (Run mode operating supply level to restore) + EXTI and RCC wake-up synchronization (4)

1. *Refer to the Cortex®-M55 system control register (SCR).*
2. *See the table "STM32N6xx vector table" in the document [1] for the list of NVIC connected peripherals.*
3. *See the table EXTI line connection in the document [1] for the list of EXTI wake-up interrupts.*
4. *See the section Power-on and wake-up sequences in the document [1].*

Table 4. Stop mode (SVOS low)

Stop mode SVOS Low	Description
Mode entry	WFI or WFE: • SLEEPDEEP = 1(1) • CPU NVIC interrupts and events cleared • ALL CPU EXTI wake-up sources cleared
	On return from ISR: • SLEEPDEEP = 1 • SLEEPONEXIT = 1(1) • CPU NVIC interrupts and events cleared • All CPU EXTI wake-up sources cleared
Mode exit	WKUP pins rising or falling edge RTC alarms (alarm A and alarm B) RTC wake-up Tamper event Timestamp event External reset in NRST pin IWDG reset
Wake-up latency	SMPS wake-up time from low-power mode (Run mode operating supply level to restore) + EXTI and RCC wake-up synchronization (2)

1. *Refer to the Cortex®-M55 system control register (SCR).*
2. *See the section Power-on and wake-up sequences in the document [1].*

HAL version

```
int main(void)
{
    SystemInit(); //Sometimes already called in the Startup file
    HAL_Init();
    SystemClock_Config();//Don't forget configuring MSI

    /* User code begin*/
        //etc...
    /* User code end*/

    /*Voltage scaling to SVOS HIGH*/
    HAL_PWREx_ControlStopModeVoltageScaling(PWR_REGULATOR_STOP_VOLTAGE_SCALE3);

    /*Allowing MSI clock enabled during Stop mode (only SVOS HIGH)*/
     __HAL_RCC_MSISTOP_ENABLE();

    /*Entering Stop mode */
    HAL_PWR_EnterSTOPMode(0, PWR_STOPENTRY_WFI);
  return (0);
}
```

4.1.3 Standby mode

The Standby mode is used to achieve the lowest power consumption. It is based on the CPU subsystem Stop mode. However, if the V_{DDCORE} supply regulator is powered off, then the core domain is consequently powered off. The PLLs, HSI, MSI, and HSE oscillators are also switched off. The SRAM and register contents are lost except for the backup domain (RTC registers, backup register, and backup SRAM), and the retention domain.

When the microcontroller exits the Standby mode, the HSI is selected as the system and kernel clock. RCC registers are reset to their initial values except for the backup domain configurations.

Warning: *The BSEC CLOCK must be enabled in RCC_APB4HENR register to be able to enter Standby mode.*

Note: All examples provided in this section must be executed in a Secure mode.

For more information, see the section Standby mode in the document [1], chapter *Clock generation in Stop and Standby modes*.

To enter or exit the Standby mode, refer to the Table 5.

Table 5. Standby mode

Standby mode	Description
Mode entry	WFI or WFE: • SLEEPDEEP = 1 (1) • CPU NVIC interrupts and events cleared • All CPU EXTI wake-up sources cleared • All WKUPFx bits of PWR_WKUPSR cleared (2) • PDDS = 1 (3)
Mode exit	WKUP pins rising or falling edge RTC alarms (alarm A and alarm B) RTC wake-up Tamper event Timestamp event External reset in NRST pin IWDG reset
Wake-up latency	System reset phase (4)

1. *Refer to the Cortex®-M55 system control register (SCR).*
2. *Refer to the PWR wake-up status register in the document [1].*
3. *Refer to the PWR CPU control register (PWR_CPUCR) in the document [1].*
4. *See the section System and application resets in the document [1].*

HAL version

```
int main(void)
{
    SystemInit(); //Sometimes already called in the Startup file
    HAL_Init();
    SystemClock_Config();

    /* User code begin*/
        //etc...
    /* User code end*/

    /* Clear standby flag */
    __HAL_PWR_CLEAR_FLAG(PWR_FLAG_SBF);

    /*Entering Standby mode */
    HAL_PWR_EnterSTANDBYMode();
  return (0);
}
```

Standby mode with retention

The I-TCM and D-TCM can be retained in Standby mode by setting the bit TCMRBSEN of the PWR_CR4 register.

The I-TCM FLEXMEM can be retained in Standby mode by setting the bit TCMFLXRBSEN of the PWR_CR4 register.

The backup SRAM can be retained in Standby mode by setting the bit BKPRBSEN of the PWR_BDCR2 register. Note that any write access to the backup domain (therefore including BKPSRAM) is controlled by the bit DBP of the PWR_DBPCR register.

An application can be retained in retention memory and executed without going into the BootROM when the MCU wakes up from the Standby mode. To do so, the user must provide to the SYSCFG_INITSVTORCR register the address of the secure vector table inside the secure application. It must be done before entering the Standby mode. For nonsecure applications, the user must provide the address of the nonsecure vector table to the SYSCFG_INITNSVTORCR register.

4.1.4 VBAT mode

Still, with the target to narrow down the power consumption, the VBAT mode is performed by powering down all power supplies. Only the backup domain is supplied from the VBAT pin. The latter is supplied from an external battery to retain its content. The switching between V_{DD} to V_{BAT} is controlled by the power-down reset (PDR) that monitors the V_{DD} supply.

4.2 Internal SMPS step-down converter in low-power mode

To perform low-power mode with the internal SMPS, the bit SDEN in the PWR_CR1 register must be set.

This bit must be written as soon as possible after the device starts, and it can be written once after the POR (power on reset).

When the V_{DDCORE} is supplied by the internal SMPS step-down converter, the supply works in three different power modes: main, low-power, or off. The converter operating modes depend on the system modes (Run/Sleep, Stop, Standby). They are configured through the associated voltage scaling VOS and SVOS levels.

See the Table 6 below.

Table 6. Operating mode versus internal SMPS supply value

Operating mode	Internal SMPS supply value
Run/Sleep mode	Main (1): • VOS high • VOS low

Operating mode	Internal SMPS supply value
Stop mode	Low power [2]: • SVOS high • SVOS low
Standby mode	Off

1. *Refer to the document [3] for the typical value of VOS high/low.*
2. *Refer to the document [3] for the typical value of SVOS high/low.*

4.3 External power supply in low-power mode

To perform low-power mode with the internal SMPS, the bit SDEN in the PWR_CR1 register must be cleared.

This bit must be written as soon as possible after the device starts, and it can be written once after the POR (power on reset).

When the V_{DDCORE} is supplied from an external source (bypass mode), the supply source can operate in Run/Sleep, Stop, or Standby mode. See the Table 7 below.

Table 7. Operating mode versus external supply value

Operating mode	External supply value
Run/Sleep mode	Main [1]: • VOS HIGH • VOS LOW
Stop mode	Low power [2]: • SVOS HIGH ONLY
Standby mode	Off

1. *Refer to the document [3] for the typical value of VOS high/low.*
2. *Refer to the document [3] for the typical value of SVOS.*

In Run mode, the external source voltage is scalable through VOS low and VOS high. The bit VOS in the PWR_VOSCR register has no effect on the external supply. User application can use this bit to track the status of the external supply. Only SVOS high is allowed in Stop mode. In Standby mode, the external regulator must be switched off. Therefore, the PWR_ON pin is used to disable the external regulator.

4.4 Control signals

When starting up, the V_{DD} and $V_{DDA18AON}$ must be present before the V_{DDCORE}. The POR monitors the V_{DD} and $V_{DDA18AON}$ supplies, and once they are above the POR threshold level, the PWR_ON is set high.

Therefore, this pin is used to enable the external voltage regulator providing the internal SMPS to supply the V_{DDCORE}. When the internal SMPS is bypassed, the PWR_ON pin is used to enable the external voltage that supplies the V_{DDCORE}. In addition, the PWR_ON pin can be used, for example, to enable the external power supply such as the $V_{DD33USB}$ or V_{DDIO4}/V_{DDIO5}.

The output signal PWR_ON can be used to inform the external circuitry that the MCU is in Standby mode. In that case, the PWR_ON is set low and the core domain is powered down.

The output signal PWR_CSLEEP can be used to inform the external circuitry that the MCU is in Sleep mode.

The output signal PWR_CSTOP can be used to inform the external circuitry that the MCU is in Stop mode.

See the Table 8 below for the output pin information.

Table 8. Output pin information

Control signals	Output pin
PWR_ON	PWR_ON
PWR_CSLEEP	PA10 (alternate function AF0)
PWR_CSTOP	PA5 (alternate function AF0)

4.5 Low-power mode V_{DDIOx} = 1.8 V

The STM32N6 can operate with V_{DDIO} = 1.8 V to save power. To have the right power consumption at 1.8 V, the HSLV_VDD bit of the OTP 124 must be fused if the V_{DDIO} is powered below 2.5 V.

The same process can be applied to:

- V_{DDIO2} (XSPIM_P1)
- V_{DDIO3} (XSPIM_P2)
- V_{DDIO4} (SDMMC1)
- V_{DDIO5} (SDMMC2)

The HSLV_VDDIO2, HSLV_VDDIO3, HSLV_VDDIO4, and the HSLV_VDDIO5 bits of the OTP 124 must be fused if these voltages are powered below 2.5 V. See Table 9.

Table 9. The OTP V_{DDIOx} descriptions

	OTP bits	Descriptions
OTP 124	HSLV_VDDIO5 [13]	0: V_{DDIO5} is above 2.5 V (default at reset) 1: V_{DDIO5} is below 2.5 V
	HSLV_VDDIO4 [14]	0: V_{DDIO4} is above 2.5 V (default at reset) 1: V_{DDIO4} is below 2.5 V
	HSLV_VDDIO3 [15]	0: V_{DDIO3} is above 2.5 V (default at reset) 1: V_{DDIO3} is below 2.5 V
	HSLV_VDDIO2 [16]	0: V_{DDIO2} is above 2.5 V (default at reset) 1: V_{DDIO2} is below 2.5 V
	HSLV_VDD [17]	0: V_{DDIO} is above 2.5 V (default at reset) 1: V_{DDIO} is below 2.5 V

Note: For further information, see the table about STM32N6xx OTP fuse description in the document [1].

These bits are shadowed and can be accessed from the BSEC_FVRw register. The secure address for the OTP 124 is 0x560091F0.

- The bit VDDIOVRSEL of the PWR_SVMCR3 register must reflect the bit HSLV_VDD of the OTP 124.
- The bit VDDIO2VRSEL of the PWR_SVMCR3 register must reflect the bit HSLV_VDDIO2 of the OTP 124.
- The bit VDDIO3VRSEL of the PWR_SVMCR3 register must reflect the bit HSLV_VDDIO3 of the OTP 124.
- The bit VDDIO4VRSEL of the PWR_SVMCR1 register must reflect the bit HSLV_VDDIO4 of the OTP 124.
- The bit VDDIO5VRSEL of the PWR_SVMCR2 register must reflect the bit HSLV_VDDIO5 of the OTP 124.

Table 10 provides details about the $_{DDIOx}$ I/Os voltage range selection.

Table 10. The V_{DDIOx} I/Os voltage range selection

	PWR bits	Descriptions
PWR_SVMCR2	VDDIO5VRSEL [24]	0: V_{DDIO5} is above 2.5 V (default at reset) 1: V_{DDIO5} is below 2.5 V
PWR_SVMCR1	VDDIO4VRSEL [24]	0: V_{DDIO4} is above 2.5 V (default at reset) 1: V_{DDIO4} is below 2.5 V
PWR_SVMCR3	VDDIO3VRSEL [26]	0: V_{DDIO3} is above 2.5 V (default at reset) 1: V_{DDIO3} is below 2.5 V
	VDDIO2VRSEL [25]	0: V_{DDIO2} is above 2.5 V (default at reset) 1: V_{DDIO2} is below 2.5 V

	PWR bits	Descriptions
PWR_SVMCR3	VDDIOVRSEL [24]	0: V_{DDIO} is above 2.5 V (default at reset) 1: V_{DDIO} is below 2.5 V

Note: If V_{DDIOx}VRSEL bit is not set and the V_{DDIOx} supply is in 1v8 range, the corresponding I/Os works in degraded mode.

For more information:

- See *PWR supply voltage monitoring control register 1* in [1].
- See *PWR supply voltage monitoring control register 2* in [1].
- See *PWR supply voltage monitoring control register 3* in [1].

Warning: *If the HSLV_VDDIOx and VDDIOxVRSEL bits are set while V_{DDIOx} is in the 3V3 range, it can damage the device.*

5 Power consumption optimization use case

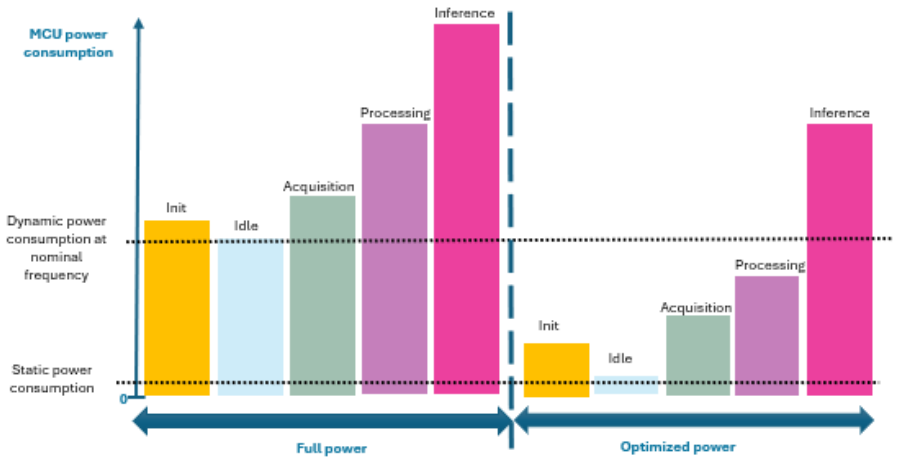
This section describes use case to optimize power consumption on all STM32N6 devices.

5.1 Power optimization methodology

In general, audio, video, or signal processing applications (with or without AI) contain identical phases: initialization phase, acquisition phase, idle phase, pre and processing phases, inference phase and more.

If each phase is running at nominal frequency, the MCU consumes more energy than strictly needed. Optimization can be done in each phase to reduce the consumption to reach as close as possible the static power consumption. (see Figure 4).

Figure 4. Full power consumption versus optimized power consumption



Some techniques can be achieved on STM32N6 to do such power consumption optimization depending on the application constraint:

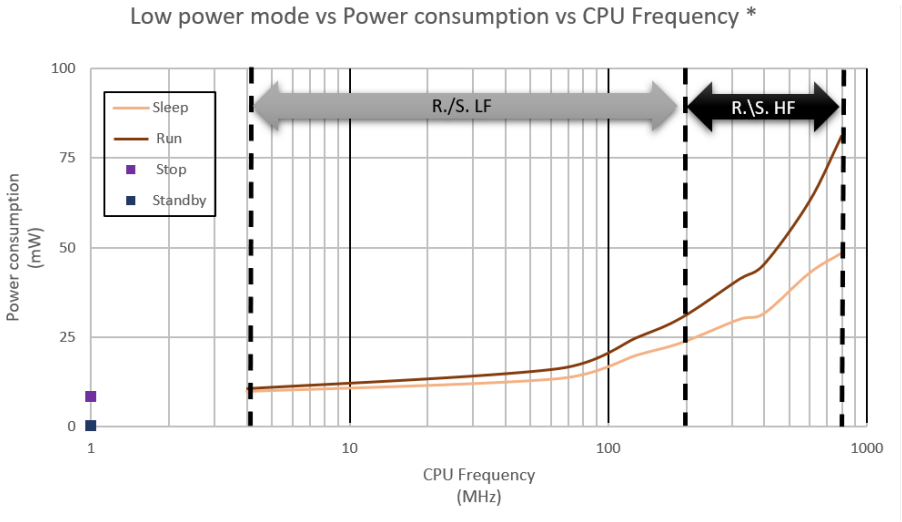
- Clock gating: technique used to reduce dynamic power dissipation by removing the clock when the circuit is not in use.
- Low-power mode (sleep, stop, standby)
- DVFS (dynamic voltage and frequency scaling): technique used to scale dynamically the frequency of clock and the supply according to the workload to balance performance and power saving.
- MDPS/DPS (memory/dynamic power switching): power down unused memory and unused features. This technique is possible because STM32N6 have independent supplies for some I/O and peripherals. Its internal memories have power switch. Furthermore, the MCU system is supplied following different power domains.

Frequency scaling

According to Figure 5, the most power effective is Standby mode. This mode should be entered whenever possible before considering Stop mode that consumes ten times higher.

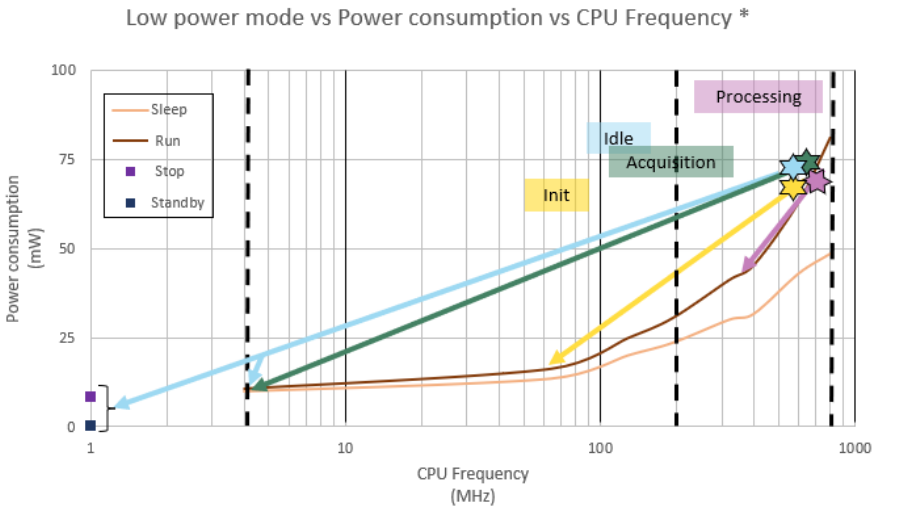
In other hand, Run/Sleep mode is spread in a wide spectrum of consumption (see Figure 5), depending on several parameters (core and bus frequency, resources). Therefore, if the application allows it, it can always apply dynamic frequency scaling to reduce the power consumption (see Figure 6).

Figure 5. STM32N6 power figures



Note: Power consumption values from document [3].

Figure 6. Optimization phases figures



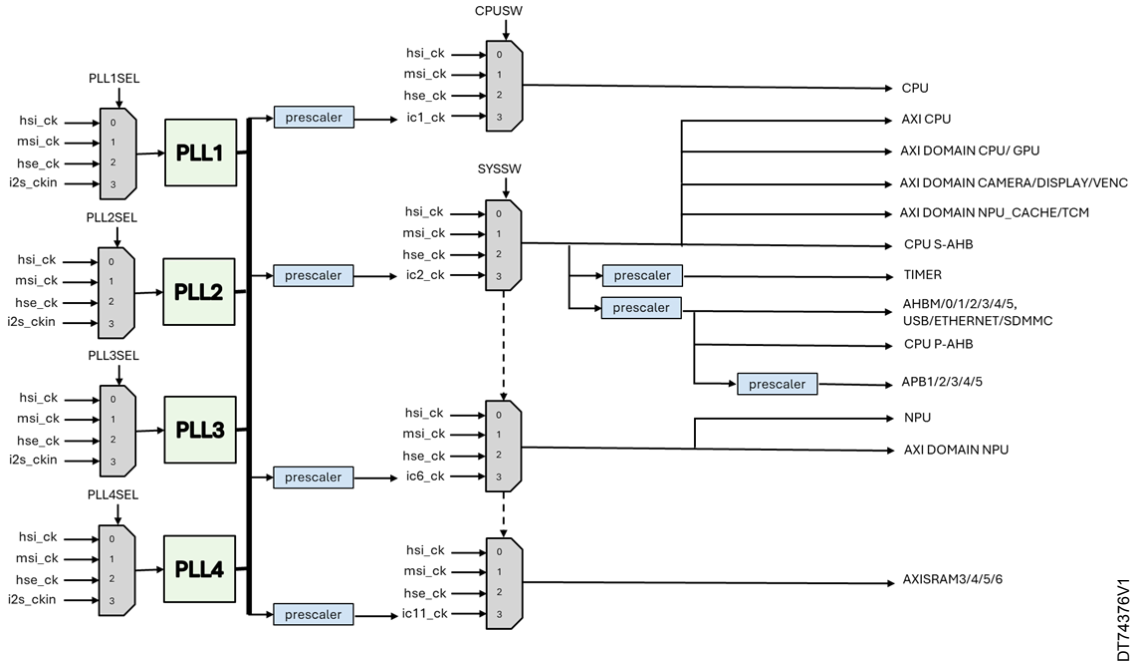
Note: Power consumption values from document [3].

Scaling and switching dynamically the clock during the execution of the application is possible thanks to clock switch, prescaler, and configurable PLL present on STM32N6 (see Figure 7).

For detailed information, refer to:

- *Functional description of RCC clocks* section in the document [1]
- *System clocks* section in the document [1]
- *PLL description* in the document [1]
- Figure 37 *Top-level clock tree* in the document [1]
- Figure 45 *Core and bus clock generation* in the document [1].

Figure 7. **Simplified clock generation**



The choice of oscillator can have an impact on consumption. HSE-based clock gives more stability and less jitter at the cost of higher consumption on $V_{DDA18AON}$. Even if an HSI frequency (64 MHz) is higher than an HSE frequency (48 MHz), HSE consumes more. Hence, to optimize power consumption, HSI, and MSI should be considered if the application constraints allow it.

The configuration of the PLL is also to be considered. A CPU running at 800 MHz consumes more than a CPU running at 600 MHz on V_{DDCORE}. If overall consumption is considered, the result might be reversed depending on how the CPU frequency is generated.

The result might be reversed in the following examples:

- Using PLL in integer mode with HSI 64 MHz as clock source
- Using PLL formula $F_{VCO} = F_{REF} \times \frac{DVIN}{DIVM}$ (see section *Using PLLs in integer mode* in the document [1])
- To have CPU at 800 MHz, the PLL needs to generate $F_{VCO} = 64\,MHz \times \frac{25}{2} = 800\,MHz$
- To have CPU at 600MHz, the PLL need to generate $F_{VCO} = 64\,MHz \times \frac{75}{4} = 1200\,MHz$, then divide this frequency by 2 with POSTDIV1 or POSTDIV2 or IC1_DIV prescaler.

From the examples above, generating 600 MHz needs higher VCO frequency. It has a large impact on the consumption of $V_{DDA18PLL}$ even if the consumption of V_{DDCORE} is less.

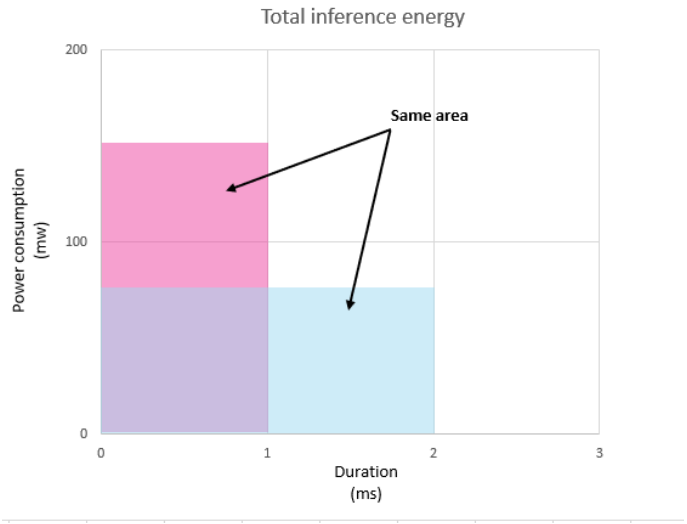
Frequency scaling optimization trade-off

Reducing the frequency may lead to lower consumption, but it also results in an increase in the duration of the task executed. At the end, the total energy consumed may not change a lot at the cost of performance loss.

Consider the power consumption of an AI inference. Suppose that one inference takes 1 ms and consumes 150 mW on average (see Figure 8). By scaling down the frequency of the CPU, the NPU, and the memory, the inference lasts 2 ms and consumes 75 mW on average. As result, the total energy consumption of both cases is the same, but in the first case the inference is faster. However, this implies the use of a power supply capable of providing such peak currents.

The user should always be aware of the total inference energy consumption and the different constraints before choosing to scale the frequency.

Figure 8. **High-frequency versus low-frequency inference energy consumption**



DT74377V1

Voltage scaling

To support the dynamic change of frequency, the voltage can also be scaled dynamically by the application. It can control external voltage supply if the STM32N6 is in external SMPS configuration (see document [3] for V_{DDCORE} maximum and minimum information). In inactive mode (Stop mode), the STM32N6 supports only SVOSHIGH value in this configuration.

In the case of internal SMPS configuration, the scaling voltage is limited:

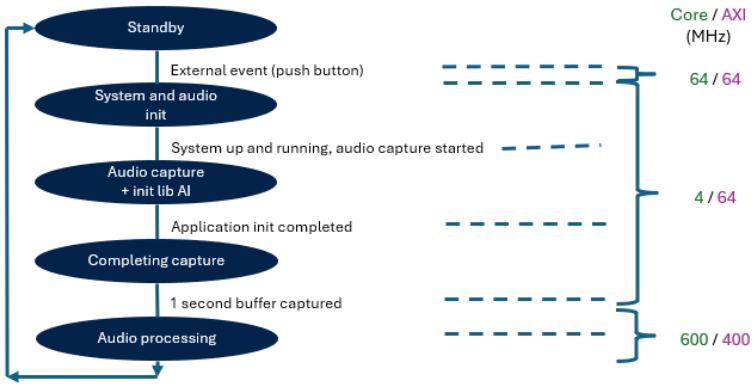
- VOSHIGH or VOSLOW in active mode (Run/Sleep mode)
- SVOSHIGH or SVOSLOW in inactive mode (Stop mode)

Note: For all next sections, audio AI processing use case is chosen as low-power case of studies, but the techniques are still valid for other use cases such as video.

5.2 Standby mode use case

The figure is an example of use case showing that Standby mode paired with dynamic frequency scaling can be possible for audio AI capture and processing to reduce the power consumption.

Figure 9. **Audio capture and processing application**



DT74378V1

5.3 Stop mode use case

This mode can replace the Standby mode in the previous use case (see Section 5.2: Standby mode use case). In this mode, no audio streaming can occur because CPU/BUS and DMA are off. The only advantage of this mode compared to the Standby mode is the reduced latency. Indeed, the CPU can wake up faster but at the cost of 10 x power penalty.

The STM32N6 has an option to leave the oscillator ON during Stop mode to further accelerate the exit from this mode. If the application is quite big, retention memory may be not enough. Therefore waking-up from standby costs the time of loading the application from non-volatile memory, whereas waking-up from Stop mode directly resume the application from internal memory.

If using the internal SMPS, the Stop mode can operate with SVOS low which reduces further the power consumption.

5.4 Run/sleep mode use case

Full power use case descriptions:

- Audio continuous acquisition
- Audio preprocessing when enough audio samples are acquired
- AI audio inference when an audio patch is ready
- All previous three steps can happen in parallel
- Audio continuous monitoring
- No event lost
- System available for any other application related or not to this use case

Every audio workload runs continuously. Power optimization is needed but the same level of functionality should be maintained.

In this implementation, 99 % of the time, the system is only acquiring and storing audio in memory. So, it is worth paying attention to this phase, by reducing the power budget close to the Stop mode.

The remaining 1 % of processing is still worth the effort, especially if AI processing complexity on the STM32N6 is expected to increase.

To achieve such optimization, the methodology previously explained in Section 5.1: Power optimization methodology is used.

Step 1: CPU/NPU dynamic clock gating

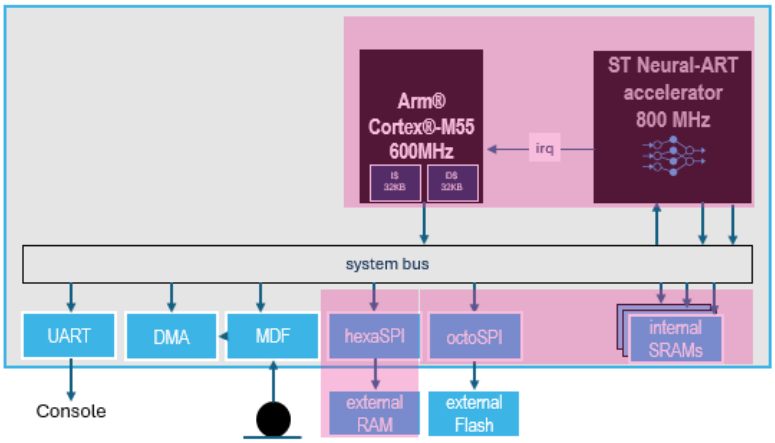
- During idle thread, the system must be put in Sleep mode (CPU is clock gated) and DMA is used to transfer data during acquisition.
- When there are no AI inferences, the NPU clock must be switched off.

Step 2: Memory dynamic power switching and dynamic power switching

- If the external RAM is unused, it must be switched off.
- If the external flash is unused with no inference, it must be switched ON only during access period.
- If the system memory is not fully used, all memory mapping must be gathered on AXISRAM1, and AXISRAM2 must be powered off.
- If NPU RAM (AXISRAM3/4/5/6) is not fully used, AXISRAM3/4/5 must be powered OFF and AXISRAM6 powered ON only during inference and audio acquisition.
- NPU CACHE must be powered only during inference.
- If the backup SRAMs is unused, it must be powered off.
- If I-TCM/D-TCM is unused, it must be switched off.

The following figure shows the resources impacted by the DPS.

Figure 10. DPS target



DT74384V1

Step 3: Dynamic voltage and frequency scaling

- Full power operating point scheme (nominal):
  - No overdrive (VOSLOW = 0.8 V)
  - CPU at 600 MHz and system at 400 MHz
  - NPU at 800 MHz and NPU_NIC at 800 MHz
  - HSI at 64 MHz, PLL1 and PLL2
- Low power operating point scheme:
  - No overdrive (VOSLOW = 0.8 V)
  - CPU at 4 MHz and system at 4 MHz
    - No NPU
    - MSI at 4 MHz, no PLLs
- Operating point power transition constraints:
  - No audio samples can be lost during transitions. The audio acquisition continuity forbids to switch from one clock scheme to another. Therefore, a unique clock scheme that cops with a low-power scheme must be used.
  - Clocking MDF with lower frequency implies non-exact audio sampling frequency so model should be retrained to recover inference accuracy slight degradation. Figure 9 shows the audio frequency error at 4 MHz.
  - The UART can be clocked on a unique low-power clocking scheme.

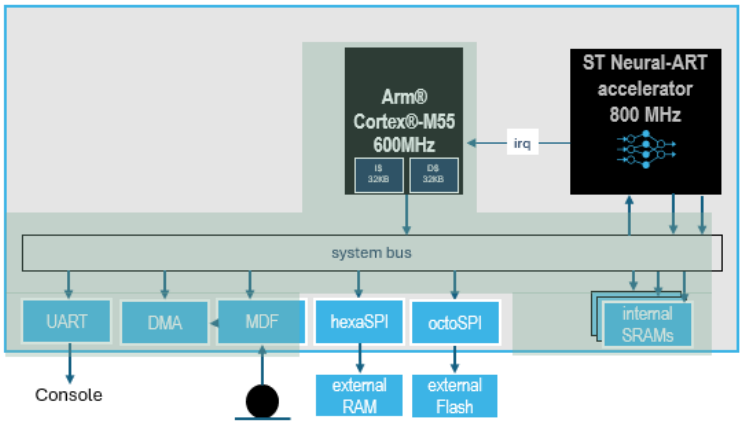
Figure 11. Audio frequency error

Config CIC + RSFLT + HPF											
Kernel clock	PROCDIV+1	CCKDIV+1	CICDEC+1	RSFLT	mdf_proc_ck	Nber of clk periods for IIR	Decimation Ratio	F_{PCM}	F_{MDF_CCK}	MDF_CCKx Duty cycle	Frequency error
MHz					MHz			kHz	MHz	%	%
4.000	1	4	16	Y	4.000	64	64	15.63	1.000	50.0	-2.3
4.000	1	5	13	Y	4.000	65	52	15.38	0.800	40.0	-3.8
4.000	1	6	10	Y	4.000	60	40	16.67	0.667	50.0	4.2
4.000	1	7	9	Y	4.000	63	36	15.87	0.571	42.9	-0.8
4.000	1	8	8	Y	4.000	64	32	15.63	0.500	50.0	-2.3

DT74380V1

The following figure shows the resources impacted by the DVFS.

Figure 12. DVFS target

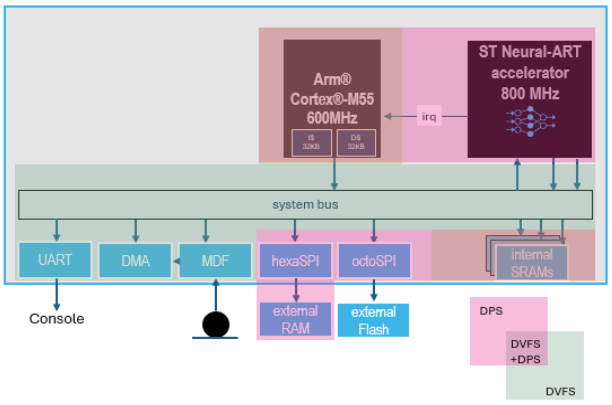


DT74385V1

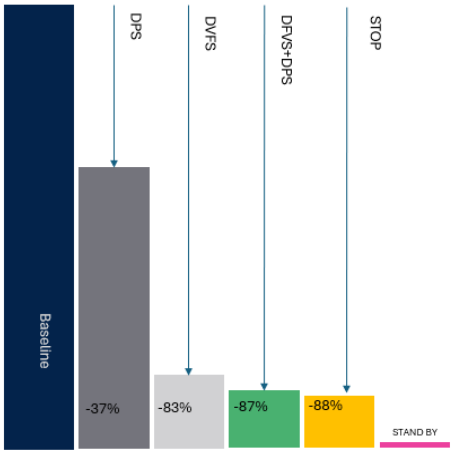
Results

87 % of power is saved by implementing power management techniques (DVFS, DPS), achieving a near stop power budget (+ 13 %). For further information, refer to Figure 13.

Figure 13. Power optimization results



	Idle	Proc	Total	vs baseline	vs Stop mode
Baseline	82.47	106.35	82.61	0%	766%
DPS	51.09	90.70	52.29	-37%	448%
DFVS	13.08	102.19	13.69	-83%	44%
DPS+DVFS	**10.21**	**89.74**	**10.78**	**-87%**	**13%**
Stop Mode	9.54	9.54	9.54	-88%	0%
Stand By	1.00	1.00	1.00	-99%	-90%



DT74382V1

6 Debug tips

This section describes how to use the debug features in low-power modes.

6.1 Enabling debugging in low-power mode

When entering a low-power mode, the clocks of the processors, the subsystem clocks, or the core power supply may be switched off. This action prevents debug access to the related unclocked domains. The STM32N6 devices support a low-power mode emulation that keeps a debug capability while in low-power mode. The DBGMCU configuration register (DBGMCU_CR) controls this through the DBG_SLEEP, DBG_STOP, and DBG_STANDBY bits.

For more information, refer to the following sections in document [1]:

- *Low-power emulation mode in RCC chapter*
- *Microcontroller debug unit (DBGMCU) in the Debug support chapter*

Note: Using low-power emulation modes has a significant impact on the power savings. Various clocks and power supplies are kept on. These modes must only be used for debugging purposes. Power consumption measurements are not relevant while using low-power mode emulation.

Table 11. HAL debug function

Function	Description
HAL_DBGMCU_EnableDBGSleepMode() HAL_DBGMCU_DisableDBGSleepMode()	Enable/disable the debug module during the Sleep mode
HAL_DBGMCU_EnableDBGStopMode() HAL_DBGMCU_DisableDBGStopMode()	Enable/disable the debug module during the Stop mode
HAL_DBGMCU_EnableDBGStandbyMode() HAL_DBGMCU_DisableDBGStandbyMode()	Enable/disable the debug module during the Standby mode

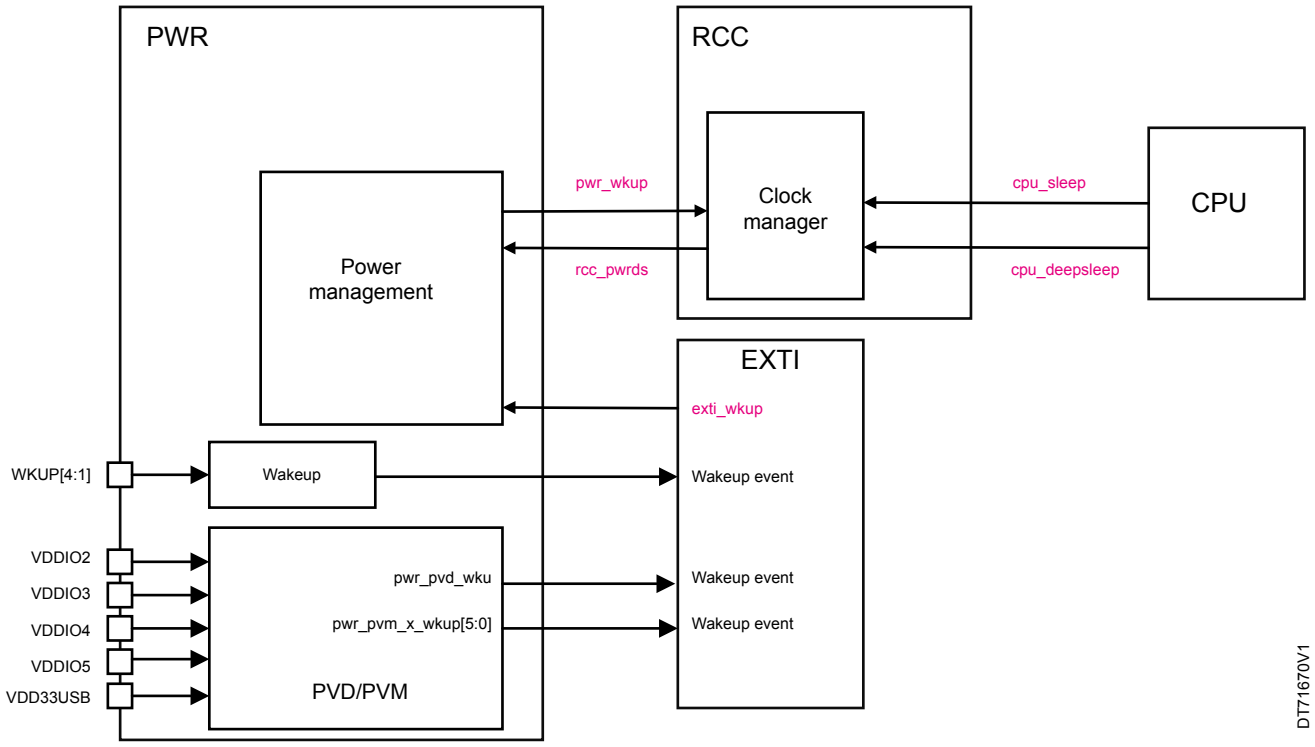
6.2 Using the hardware debug port

The hardware debug port (HDP) allows monitoring the internal signals that can be used to debug the entry and the exit from low-power modes. Those signals can be output on some pins of the STM32N6 devices. They are easily monitored through a scope, or a logic analyzer. See the table in *Mixing of HDP signals in the Hardware Debug Port* section in the document [1] for the comprehensive list of signals and their mixing configuration.

The HDP and associated GPIOs must be disabled in the final application in order to reach a lower power consumption.

Figure 14 illustrates the important internal signals and the hierarchy that the system follows when entering or exiting low-power modes.

Figure 14. **Low-power mode system hierarchy and internal signals**



The most important signals for the power-mode debugging are described in the Table 12, Table 13, and Table 14 below.

Table 12. **Monitoring the Sleep mode**

Signal	Description
CPU_SLEEPING (HDP4, HDP_MUX = 1)	This signal informs the RCC whether the CPU is in WFI or WFE state.

Table 13. **Monitoring the Stop mode**

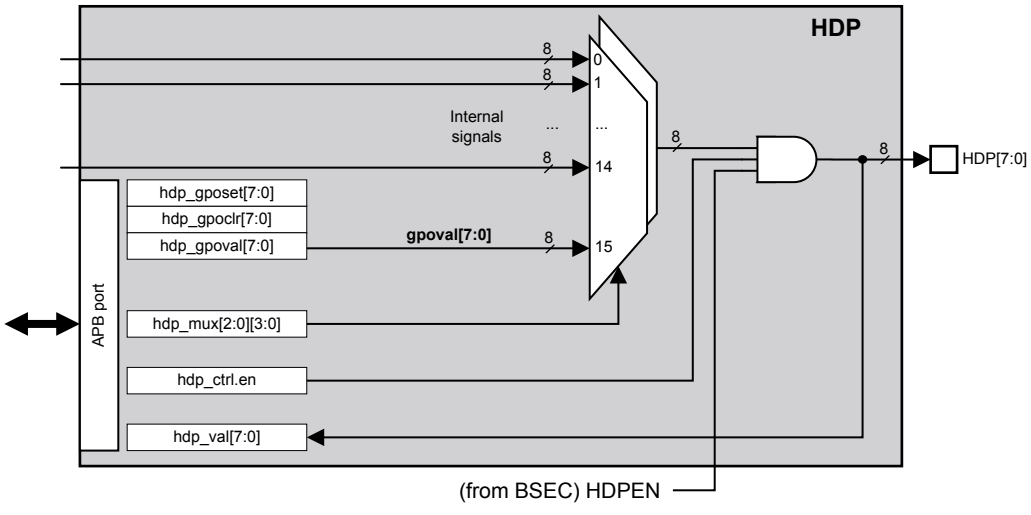
Signal	Description
CPU_SLEEPDEEP (HDP0, HDP_MUX = 1)	This signal can be used to monitor the MCU Stop modes
RCC pwrds_sys (HDP0, HDP_MUX = 6)	This signal informs the PWR that the RCC has stopped all clocks. Then, the PWR can go to Stop mode or Standby mode. It can be used to monitor the entry in Stop mode.

Table 14. **Monitoring the exit from Stop mode**

Signal	Description
EXTI1 sys_wakeup (HDP1, HDP_MUX = 5)	This signal can be used to monitor the wake-up from EXTI.

Each value that is outputted in I/Os ports can be accessed at the HDP_VAL[0:7] registers (see Figure 15).

Figure 15. **HDP**



DT71130V1

See below the HDP initialization sample code.

```
void HDP_GPIOConfig(void) {
    GPIO_InitTypeDef GPIO_InitStruct;
      /*
       * HDP   AF15
       *  0    PA8
       *  1    PA9
       *  2    PA10
       *  3    PA11
       *  4    PA12
       *  5    PA13
       *  6    PA14
       *  7    PA15
       */

      /* Enable GPIOG clock*/
      __HAL_RCC_GPIOA_CLK_ENABLE();

      /*** Route HDP outputs to GPIOs ***/
      /* AF15,[PA8:PA15] => [HDP0:HDP7] */
      GPIO_InitStruct.Pin = GPIO_PIN_8|GPIO_PIN_9|GPIO_PIN_10|
      GPIO_PIN_11|GPIO_PIN_12|GPIO_PIN_13|GPIO_PIN_14|GPIO_PIN_15;
      GPIO_InitStruct.Mode = GPIO_MODE_AF_PP;
      GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_HIGH;
      GPIO_InitStruct.Pull = GPIO_NOPULL;
      GPIO_InitStruct.Alternate = GPIO_AF15_HDP;
      HAL_GPIO_Init(GPIOA, &GPIO_InitStruct);

}
void HDP_MuxConfig(HDP_MuxInitTypeDef *HDP_MuxInitStruct)
{

  /* Clock HDP */
  __HAL_RCC_HDP_CLK_ENABLE();

  /*** Select signals for HDPx outputs ***/
  MODIFY_REG(HDP->MUX, HDP_MUX0, ((uint32_t)HDP_MuxInitStruct->HDP0));
  // Select signal on the HDP0 output
  MODIFY_REG(HDP->MUX, HDP_MUX1, ((uint32_t)HDP_MuxInitStruct->HDP1) <<   4);
  // Select signal on the HDP1 output
  MODIFY_REG(HDP->MUX, HDP_MUX2, ((uint32_t)HDP_MuxInitStruct->HDP2) <<   8);
  // Select signal on the HDP2 output
  MODIFY_REG(HDP->MUX, HDP_MUX3, ((uint32_t)HDP_MuxInitStruct->HDP3) <<  12);
  // Select signal on the HDP3 output
  MODIFY_REG(HDP->MUX, HDP_MUX4, ((uint32_t)HDP_MuxInitStruct->HDP4) <<  16);
  // Select signal on the HDP4 output
  MODIFY_REG(HDP->MUX, HDP_MUX5, ((uint32_t)HDP_MuxInitStruct->HDP5) <<  20);
  // Select signal on the HDP5 output
  MODIFY_REG(HDP->MUX, HDP_MUX6, ((uint32_t)HDP_MuxInitStruct->HDP6) <<  24);
  // Select signal on the HDP6 output
  MODIFY_REG(HDP->MUX, HDP_MUX7, ((uint32_t)HDP_MuxInitStruct->HDP7) <<  28);
  // Select signal on the HDP7 output
```

```
/* Enable HDP */
SET_BIT(HDP->CTRL, 0x1UL);

}
int main(void)
{
    HDP_GPIOConfig();

    HDP_MuxInitStruct.HDP0 = HDP0_CPU_SLEEPDEEP;
    HDP_MuxInitStruct.HDP1 = HDP1_EXTI1_SYS_WAKEUP;
    HDP_MuxInitStruct.HDP2 = HDP2_GPOVAL_2;
    HDP_MuxInitStruct.HDP3 = HDP3_GPOVAL_3;
    HDP_MuxInitStruct.HDP4 = HDP4_CPU_SLEEPING;
    HDP_MuxInitStruct.HDP5 = HDP5_GPOVAL_5;
    HDP_MuxInitStruct.HDP6 = HDP6_GPOVAL_6;
    HDP_MuxInitStruct.HDP7 = HDP6_GPOVAL_7;

    HDP_MuxConfig(&HDP_MuxInitStruct);

    HDP->GPOVAL |= 0xEC;
    return 0;
}
```

Revision history

Table 15. **Document revision history**

Date	Version	Changes
14-Nov-2024	1	Initial release.

Contents

1 General information 2
2 Glossary 3
3 STM32N6 power supply 5
3.1 System architecture 5
3.2 System supply 5
3.3 Power management 7
4 STM32N6 low-power features 8
4.1 Entering/exiting low-power modes 8
4.1.1 Sleep mode 8
4.1.2 Stop mode 9
4.1.3 Standby mode 11
4.1.4 VBAT mode 13
4.2 Internal SMPS step-down converter in low-power mode 13
4.3 External power supply in low-power mode 14
4.4 Control signals 14
4.5 Low-power mode V_{DDIOx} = 1.8 V 15
5 Power consumption optimization use case 17
5.1 Power optimization methodology 17
5.2 Standby mode use case 20
5.3 Stop mode use case 21
5.4 Run/sleep mode use case 21
6 Debug tips 24
6.1 Enabling debugging in low-power mode 24
6.2 Using the hardware debug port 24
Revision history 29

List of figures

Figure 1. High-level system architecture . . . 5
Figure 2. Retention domain . . . 6
Figure 3. Backup domain . . . 7
Figure 4. Full power consumption versus optimized power consumption . . . 17
Figure 5. STM32N6 power figures . . . 18
Figure 6. Optimization phases figures . . . 18
Figure 7. Simplified clock generation . . . 19
Figure 8. High-frequency versus low-frequency inference energy consumption . . . 20
Figure 9. Audio capture and processing application . . . 20
Figure 10. DPS target . . . 22
Figure 11. Audio frequency error . . . 22
Figure 12. DVFS target . . . 23
Figure 13. Power optimization results . . . 23
Figure 14. Low-power mode system hierarchy and internal signals . . . 25
Figure 15. HDP . . . 26

List of tables

Table 1. Glossary . . . 3
Table 2. Sleep mode . . . 8
Table 3. Stop mode (SVOS high) . . . 9
Table 4. Stop mode (SVOS low) . . . 10
Table 5. Standby mode . . . 12
Table 6. Operating mode versus internal SMPS supply value . . . 13
Table 7. Operating mode versus external supply value . . . 14
Table 8. Output pin information . . . 14
Table 9. The OTP V_{DDIOx} descriptions . . . 15
Table 10. The V_{DDIOx} I/Os voltage range selection . . . 15
Table 11. HAL debug function . . . 24
Table 12. Monitoring the Sleep mode . . . 25
Table 13. Monitoring the Stop mode . . . 25
Table 14. Monitoring the exit from Stop mode . . . 25
Table 15. Document revision history . . . 29

IMPORTANT NOTICE – READ CAREFULLY

STMicroelectronics NV and its subsidiaries (“ST”) reserve the right to make changes, corrections, enhancements, modifications, and improvements to ST products and/or to this document at any time without notice. Purchasers should obtain the latest relevant information on ST products before placing orders. ST products are sold pursuant to ST’s terms and conditions of sale in place at the time of order acknowledgment.

Purchasers are solely responsible for the choice, selection, and use of ST products and ST assumes no liability for application assistance or the design of purchasers’ products.

No license, express or implied, to any intellectual property right is granted by ST herein.

Resale of ST products with provisions different from the information set forth herein shall void any warranty granted by ST for such product.

ST and the ST logo are trademarks of ST. For additional information about ST trademarks, refer to www.st.com/trademarks. All other product or service names are the property of their respective owners.

Information in this document supersedes and replaces information previously supplied in any prior versions of this document.

© 2024 STMicroelectronics – All rights reserved